

HashMap — Internal Working & Deep Dive

Java Backend Interview Reference

Internal Constants

```
// java.util.HashMap key constants static final int DEFAULT_INITIAL_CAPACITY = 1 << 4; // 16 static final int
MAXIMUM_CAPACITY = 1 << 30; // ~1 billion static final float DEFAULT_LOAD_FACTOR = 0.75f; // resize when 75%
full static final int TREEIFY_THRESHOLD = 8; // chain -> Red-Black Tree when bucket has 8+ nodes static final
int UNTREEIFY_THRESHOLD = 6; // tree -> chain on shrink static final int MIN_TREEIFY_CAPACITY = 64; // treeify
only if table.length >= 64 transient Node<K,V>[] table; // the bucket array transient int size; // number of
key-value mappings int threshold; // next resize at: threshold = capacity * loadFactor final float loadFactor;
```

Node Structure

```
// Each bucket slot holds a linked list of Nodes (or TreeNodes in Java 8+) static class Node<K,V> implements
Map.Entry<K,V> { final int hash; // cached hash of key (avoids recomputing) final K key; V value; Node<K,V>
next; // pointer to next node in same bucket (chaining) } // Java 8+: When bucket chain >= 8 AND table.length
>= 64, converts to Red-Black Tree: static final class TreeNode<K,V> extends LinkedHashMap.Entry<K,V> {
TreeNode<K,V> parent, left, right, prev; boolean red; // O(log n) get/put instead of O(n) for long chains }
```

Hash Function — How Key is Hashed

```
// Step 1: call key.hashCode() (Object method, overridden by String, Integer, etc.) // Step 2: spread high
bits into low bits (reduce collision for poor hashCodes) static final int hash(Object key) { int h; return
(key == null) ? 0 : (h = key.hashCode()) ^ (h >> 16); // XOR upper 16 bits into lower 16 bits // Ensures
upper bits participate in bucket index calculation } // Step 3: find bucket index int index = (n - 1) & hash;
// n = table.length (always power of 2) // Equivalent to: hash % n (but faster – bitwise AND works when n is
power of 2) // Example: n=16, hash=0b...10110101 // (16-1) = 0b1111 -> index = last 4 bits of hash = 0101 = 5
```

put() Operation — Step by Step

```
// map.put("key", "value") // 1. Compute hash: h = hash("key") // 2. If table is null, initialize
(capacity=16) // 3. Compute bucket index: i = (n-1) & h // 4. Check bucket: // a. EMPTY: create new Node,
place at table[i]. Done. // b. KEY EXISTS (same hash AND key.equals()): update value. Done. // c. TREENODE:
call tree.putTreeVal(). O(log n) // d. CHAIN: traverse linked list: // - If key found: update value. Done. //
- If end of chain: append new Node. // If chain now >= TREEIFY_THRESHOLD(8) AND table.length >= 64: treeify
bucket // 5. size++. If size > threshold (capacity * 0.75): RESIZE // Simplified source view: final V
putVal(int hash, K key, V value, boolean onlyIfAbsent, boolean evict) { Node<K,V>[] tab; Node<K,V> p; int n,
i; if ((tab = table) == null || (n = tab.length) == 0) n = (tab = resize()).length; if ((p = tab[i = (n - 1) &
hash]) == null) // empty bucket tab[i] = newNode(hash, key, value, null); else { /* collision handling:
traverse chain or tree */ if (++size > threshold) resize(); return null; }
```

get() Operation — Step by Step

```
// map.get("key") // 1. Compute hash: h = hash("key") // 2. Compute bucket index: i = (n-1) & h // 3. Check
table[i]: // a. NULL: return null (key not present) // b. FIRST NODE matches (hash == node.hash &&
key.equals(node.key)): return node.value // c. TREENODE: binary search in Red-Black Tree. O(log n) // d.
CHAIN: traverse until key found or null. O(k) where k=chain length final Node<K,V> getNode(int hash, Object
key) { Node<K,V>[] tab; Node<K,V> first, e; int n; K k; if ((tab = table) != null && (n = tab.length) > 0 &&
(first = tab[(n - 1) & hash]) != null) { if (first.hash == hash && // always check first node ((k = first.key)
== key || (key != null && key.equals(k)))) return first; if ((e = first.next) != null) { if (first instanceof
TreeNode) // tree lookup return ((TreeNode<K,V>)first).getTreeNode(hash, key); do { // chain traversal if
(e.hash == hash && ((k = e.key) == key || key.equals(k))) return e; } while ((e = e.next) != null); } } return
null; }
```

Resize / Rehash

```
// Triggered when: size > capacity * loadFactor (default: 16 * 0.75 = 12 entries) // New capacity = old
capacity * 2 (always keeps power-of-2) // New threshold = new capacity * loadFactor // ALL existing entries
must be rehashed and moved to new positions // Java 8 optimization: because capacity doubles, each entry
either: // stays at same index (high bit of hash = 0), OR // moves to old_index + old_capacity (high bit of
hash = 1) // No need to recompute full hash! // Example: capacity 16 -> 32. Entry at index 5: // bit 4 of hash
= 0: stays at index 5 // bit 4 of hash = 1: moves to index 5 + 16 = 21 // Resize timeline (default load factor
0.75): // capacity=16, resize at 12 entries // capacity=32, resize at 24 entries // capacity=64, resize at 48
entries (also enables treeification) // capacity=128, resize at 96 entries ... and so on
```

Java 8 Treeification — LinkedList to Red-Black Tree

Condition	Detail
Treeify trigger	Bucket chain length $\geq$ TREEIFY_THRESHOLD (8) AND table.length $\geq$ MIN_TREEIFY_CAPACITY (64)
Why 64 table size minimum	If table is small, a resize would redistribute entries and reduce chain lengths. Treeifying small tables wastes memory and causes premature optimization.
Untreeify trigger	During resize, if tree has $\leq$ UNTREEIFY_THRESHOLD (6) nodes, it converts back to a linked list
Time complexity improvement	Linked list: $O(n)$ worst case for get/put in one bucket. Red-Black Tree: $O(\log n)$. Critical for DoS protection against hash flooding attacks.
Memory cost	TreeNode is $\sim 2x$ size of Node. Only used when chain is long — typical well-distributed maps never treeify.

Collision — What Happens

Collision occurs when two different keys hash to the same bucket index. HashMap handles this with separate chaining (linked list, or tree in Java 8+).

```
// Example of collision: map.put("Aa", 1); // "Aa".hashCode() = 2112 map.put("BB", 2); // "BB".hashCode() = 2112 <- SAME hash! // Both land in same bucket. "Aa" is first node; "BB" added as next node. // get("BB"): go to bucket, first node key="Aa" (hash matches but !equals) -> check next -> "BB" found. // Deliberate collision attack (pre-Java 8): // Attacker sends many keys with same hashCode -> all in one bucket ->  $O(n)$  per lookup -> DoS // Fix: Red-Black Tree in Java 8 limits worst case to  $O(\log n)$  // Also: String has hash randomization option (-XX:+UseStringDeduplication)
```

Time Complexity

Operation	Average Case	Worst Case (Java 7)	Worst Case (Java 8+)
get(key)	$O(1)$	$O(n)$ — all keys in one bucket	$O(\log n)$ — tree bucket
put(key, value)	$O(1)$	$O(n)$	$O(\log n)$
remove(key)	$O(1)$	$O(n)$	$O(\log n)$
containsKey	$O(1)$	$O(n)$	$O(\log n)$
containsValue	$O(n)$ always	$O(n)$	$O(n)$ — must scan all buckets
resize	$O(n)$ — amortized $O(1)$ per put	$O(n)$	$O(n)$

HashMap vs Other Map Implementations

Map	Ordering	Null Keys	Thread Safe	Performance	Use Case
HashMap	None	1 null key	No	$O(1)$ avg	General purpose, best performance
LinkedHashMap	Insertion order (or access order)	1 null key	No	$O(1)$ avg, slight overhead	LRU cache, ordered iteration
TreeMap	Natural/Comparator sorted order	No null keys	No	$O(\log n)$	Sorted keys, range queries (subMap, headMap)
Hashtable	None	No null keys or values	Yes (synchronized)	Slow (coarse lock)	Legacy — don't use; use ConcurrentHashMap
ConcurrentHashMap	None	No null keys or values	Yes (segment/bucket lock)	$O(1)$ avg, high concurrency	Multi-threaded environments

Map	Ordering	Null Keys	Thread Safe	Performance	Use Case
EnumMap	Enum declaration order	No	No	Fastest — array-backed	Enum keys only
WeakHashMap	None	1 null key	No	O(1) avg	Cache where entries expire with key GC

ConcurrentHashMap Internals

```
// Java 8 ConcurrentHashMap: NO null keys/values // Uses bucket-level synchronization (not full table lock like Hashtable) // Each bucket: CAS for empty slot, synchronized for non-empty // Default concurrency level: up to 16 simultaneous write threads (one per bucket set) // Key operations: // put(): CAS on empty bucket, synchronized(bucket_head) on occupied bucket // get(): FULLY LOCK-FREE -- reads are volatile/atomic, never blocked // size(): LongAdder-based count -- avoids contention on single counter // computeIfAbsent (safe for concurrent initialization): cache.computeIfAbsent(key, k -> expensiveCompute(k)); // Atomic: compute only once even with concurrent calls for same key
```

hashCode() and equals() Contract

HashMap correctness depends entirely on proper hashCode/equals implementation. The contract:

Rule	Detail
Consistency	hashCode() must return same value for same object across calls in same JVM run
Equality implies same hash	If a.equals(b) is true, then a.hashCode() == b.hashCode() MUST be true
Unequal objects CAN share hash	a.hashCode() == b.hashCode() does NOT mean a.equals(b) — this is collision, not a contract violation
Violation consequence	If you override equals() without overriding hashCode(), two "equal" objects land in different buckets — map.get() returns null for logically equal keys
@Override both	Always override both in the same class. IDEs auto-generate them. Java 7+: use Objects.hash(field1, field2, ...)

```
// BAD: overrides equals but not hashCode class Point { int x, y; @Override public boolean equals(Object o) { return ((Point)o).x==x && ((Point)o).y==y; } // No hashCode -> uses Object.hashCode() -> memory address -> different for equal points! } map.put(new Point(1,2), "A"); map.get(new Point(1,2)); // returns NULL -- different object, different hash! // GOOD: @Override public int hashCode() { return Objects.hash(x, y); }
```

Common Pitfalls

Pitfall	Problem	Fix
Mutable key	If key object mutates after put(), its hashCode() changes. get() looks in wrong bucket. Entry lost.	Use immutable objects as keys (String, Integer, enums). If must use mutable key, never modify it while in map.
Poor hashCode	All keys return same hashCode -> all in one bucket -> O(n) performance. E.g., hashCode() { return 1; }	Distribute well: use Objects.hash(), include all distinguishing fields
Missing null check	TreeMap.put(null, val) throws NullPointerException. ConcurrentHashMap also rejects null.	Know which map accepts null. HashMap: 1 null key allowed. TreeMap, ConcurrentHashMap: no null keys.
Thread-safety	Two threads put() simultaneously -> race on same bucket -> data loss, infinite loop (Java 6 resize bug)	Use ConcurrentHashMap or Collections.synchronizedMap(new HashMap<>())
Initial capacity for known size	Default 16 + resize at 12 entries. Loading 1000 entries causes ~6 resizes.	new HashMap<>((int)(expectedSize / 0.75) + 1) to prevent resizing

LinkedHashMap — LRU Cache Pattern

```
// LinkedHashMap maintains doubly-linked list across all entries // accessOrder=true: most recently accessed entry moves to tail // Override removeEldestEntry to auto-evict on size limit public class LRUCache<K,V> extends LinkedHashMap<K,V> { private final int maxSize; public LRUCache(int maxSize) { super(maxSize, 0.75f,
```

```
true); // accessOrder=true this.maxSize = maxSize; } @Override protected boolean
removeEldestEntry(Map.Entry<K,V> eldest) { return size() > maxSize; // evict least-recently-used when full } }
LRUCache<Integer, String> cache = new LRUCache<>(100); cache.put(1, "A"); cache.get(1); // moves 1 to tail
(most recent) // When size > 100, eldest (least recently used) is auto-removed
```

Interview Q&A;

Q1: What is the default initial capacity and load factor of HashMap?

Default initial capacity is 16 (power of 2) and default load factor is 0.75. The map resizes when size exceeds capacity × loadFactor (16 × 0.75 = 12 entries). After resize, capacity doubles: 16 → 32 → 64 → 128...

Q2: Why must HashMap capacity always be a power of 2?

To compute bucket index, HashMap uses (n-1) & hash instead of hash % n. This bitwise AND is ~5x faster than modulo, but only works correctly when n is a power of 2 (because n-1 becomes a bitmask of all 1s in the lower bits). If you pass a non-power-of-2 capacity to the constructor, HashMap rounds it up to the next power of 2.

Q3: What happens when two keys have the same hashCode?

They collide and are placed in the same bucket. In Java 7: a linked list (chain) stores both nodes; get() traverses the chain using equals() to find the right key — O(n). In Java 8+: if the chain reaches 8 nodes AND table length ≥ 64, the chain converts to a Red-Black Tree — O(log n). This prevents DoS via hash flooding.

Q4: Why does ConcurrentHashMap not allow null keys or values?

In a concurrent context, if get() returns null, you cannot tell whether the key is absent or mapped to null — you'd need a second containsKey() call, but another thread could modify the map between the two calls, making the check unreliable (a race condition). HashMap allows null because it's single-threaded and you can check containsKey() safely.

Q5: What is the difference between HashMap and Hashtable?

Hashtable is legacy (Java 1.0), synchronized on every method (coarse lock), does not allow null keys/values, and is slower. HashMap is not synchronized, allows one null key, and is faster. For thread safety, use ConcurrentHashMap (not Hashtable) — it uses fine-grained bucket-level locking and allows fully lock-free reads.

Q6: How does Java 8 optimize HashMap resize?

In Java 8, since capacity always doubles, each entry's new index is either the same as before or old_index + old_capacity. This is determined by checking one specific bit of the hash (the bit at position old_capacity). No need to recompute the full hash — just check that one bit. This makes resize more cache-friendly and slightly faster than Java 7.

Q7: What is hash collision and how does HashMap handle it in Java 8?

Hash collision is when two keys produce the same bucket index. HashMap handles collisions with separate chaining: each bucket holds a linked list of nodes. In Java 8, if a bucket's chain exceeds 8 nodes (and the table has ≥ 64 slots), the list is converted to a Red-Black Tree for O(log n) operations. When the tree shrinks to 6 nodes (during resize), it converts back to a list.

Q8: What is the internal structure of a HashMap Node?

Each node is an instance of HashMap.Node<K,V> containing: final int hash (cached hash of the key), final K key, V value, and Node<K,V> next (pointer to next node in the chain for collision handling). In Java 8, TreeNode extends LinkedHashMap.Entry and adds tree pointers (parent, left, right, prev) and a boolean red for Red-Black Tree coloring.

Q9: Why is String a good key for HashMap?

String is immutable (hashCode cannot change after put), caches its hashCode after first computation (performance), and has a well-distributed hashCode implementation. Immutability is the critical property — mutable keys can change their hashCode after insertion, making the entry unreachable (it's in the wrong bucket for its new hash).

Q10: How do you create a thread-safe LinkedHashMap for LRU cache?

Extend LinkedHashMap with accessOrder=true and override removeEldestEntry() to return true when size exceeds maxSize. Wrap with Collections.synchronizedMap() for thread safety. For high-concurrency scenarios, prefer Caffeine cache (which provides thread-safe LRU/LFU/W-TinyLFU eviction with much better performance than synchronized LinkedHashMap).